Lab 3: Serve and measure

What does it cost to run your model?

Aarne Talman

12 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

The question

Lab 3

How fast does your model answer, and what do you give up to make it faster?

You serve your Lab 2 model with **SGLang**, an inference engine, and measure:

- **speed** with 1, 8 and 32 requests at the same time;
- **bf16 versus FP8**: is 8-bit faster, and does it change the answers on your Lab 2 test set?
- **valid output**: how often JSON answers are broken, with and without a JSON schema.

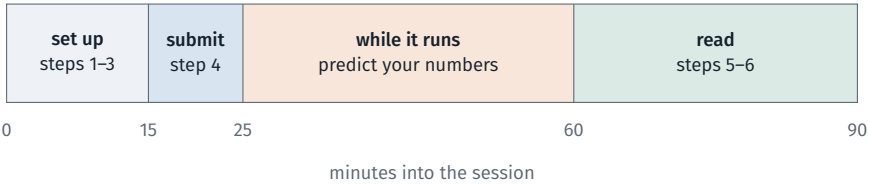
For the final report

- three tables (`summarize_results.py` prints them)
- a short recommendation: bf16 or FP8, with or without a schema, and why

Minimum: README steps 1–6 once, with your Lab 2 task.

Everything you need is in `Week-3/Lab-3/README.md`, step by step. Nothing is trained today.

Today



Start with the **smoke test** (step 2): it checks the scripts with a fake server, no GPU. If it has not passed by minute 15, put your hand up.

While the job runs, predict your tokens per second with one user: GPU memory speed (4 TB/s) divided by the model size (about 1.6 GB for Qwen3.5-0.8B in bf16) gives an upper limit. Afterwards, compare it with the measurement.

You do not have to finish today: **continue on your own time**. Work in an SSH session on `roihu-gpu.csc.fi`.

One GPU job does everything

You submit **one job** (README step 4) with your Lab 2 config. It runs the steps below for bf16 and then for FP8, and prints three tables.

1. Start the server	SGLang loads the model, in bf16 or FP8, and waits for requests
2. Speed test	20 warm-up requests (not counted), then 64 requests each with 1, 8 and 32 at the same time; every answer is exactly 128 tokens long
3. Test set	every Lab 2 test example, with the Lab 2 prompt and scorer
4. JSON (bf16 only)	the test set again, asking for `{"label": ...}`, once without and once with a JSON schema
5. Stop the server	then the same for FP8

Only one thing changes between bf16 and FP8: the precision of the weights. Model, test set, prompt and settings stay the same.

What the numbers mean

Number	What it is	Who notices
time to first token	waiting and reading the prompt, until the answer starts	a user looking at an empty box
time per output token	the gap between later tokens	a user reading the answer
throughput	output tokens per second for all requests together	whoever pays for the GPU

What to expect: with more requests at the same time, throughput goes up, because the GPU serves several users per step, but each user waits a little longer per token.

The tables show the **median** (the typical request) and the raw files also have **p95** (the slowest one in twenty).

Reading the results honestly

FP8: report both sides

- gained: tokens per second, latency
- lost: accuracy points on your test set

With a small model such as Qwen3.5-0.8B, FP8 may not be faster at all: that is a valid finding. `show_differences.py` lists the test items where bf16 and FP8 disagree.

JSON: report both numbers

- invalid outputs, without and with the schema
- accuracy, without and with the schema

A schema can bring invalid outputs to zero and still lower accuracy. If that happens, it belongs in your recommendation.

Small differences can be chance

One test example is worth $100/n$ points, and repeated speed runs differ a little. Do not read much into a difference of a point or two.

Checklist for today

[ ] files updated with `git pull` (step 1)
[ ] smoke test passed (step 2)
[ ] SGLang found (step 3)
[ ] job submitted with your Lab 2 config (step 4)
[ ] prediction written down: tokens/s with one user
[ ] tables printed with `summarize_results.py` (step 5)
[ ] differences read with `show_differences.py`
[ ] results folder and configs kept (step 6)

Good practice on Roihu

- follow a job with `squeue --me` and `tail -f logs/lab3-<jobid>.out`
- do not run models on the login node
- a running server keeps its GPU: stop it and leave interactive sessions with `exit`

Stuck for more than ten minutes? Put your hand up. The README has a troubleshooting table.